

ELL305 Computer Architecture

Term Paper — Draft #1 (v0.1)

Progress Report: Work Done as of 24 August 2026

[Your Name], [Entry Number]
[Partner Name], [Entry Number]

24 August 2026

1 Work Done as of 24 August 2026

1.1 Chosen Topic

The chosen topic is **memory consistency models** in shared-memory multiprocessors.

Working title.

Revisiting Sequential Consistency: Memory Consistency Models, Their Cost, and Whether Speculation Has Made Strong Ordering Affordable

Scope. A shared-memory multiprocessor must specify when, and in what order, one core's writes become visible to the others. That specification is the machine's memory consistency model, and it is an architectural contract as binding as the instruction set. The paper covers the design space from sequential consistency through TSO, PSO, weak ordering and release consistency; the mechanisms that make relaxation necessary (store buffers, out-of-order execution) and those that could make it unnecessary (speculative execution with coherence-based detection); and the historical argument that led the industry to reject sequential consistency around 1990.

Guiding question. Sequential consistency was rejected as a hardware memory model on performance grounds in the late 1980s, on machines that did not speculate. A modern out-of-order core already contains both mechanisms a strong model requires — coherence invalidations that announce remote writes, and a squash-and-restart path built for branch misprediction recovery. The paper asks whether the performance objection still holds, and, if it does not, what else accounts for the continued absence of sequentially consistent hardware.

The work is organised around four questions:

Q1 Formal

Can SC, TSO, PSO, weak ordering and release consistency be expressed as successively weaker constraints on a single set of ordering relations, with strict containment proved rather than asserted?

Q2 Empirical

Which relaxations are *observable* on real x86 and ARM hardware, as distinct from those merely *permitted* by the architecture?

Q3 Quantitative

Holding the microarchitecture fixed and varying only the memory model, what does sequential consistency cost with and without speculative load execution, as core count rises?

Q4 Interpretive

If speculative SC proves inexpensive, what accounts for its absence from shipping processors?

1.2 Mapping to the Prescribed Textbook

The prescribed text is S.R. Sarangi, *Basic Computer Architecture*, version 3.09 [22]. The topic maps principally to **Chapter 12, *Multiprocessor Systems***, and within it to **§12.4, *MIMD Multiprocessors***.

§	Title (page)	Role in the paper
Primary — Chapter 12		
12.4.1	Logical Point of View (579)	Shared-memory abstraction.
12.4.2	Coherence (581)	The coherence/consistency distinction, developed at length.
12.4.3	Memory Consistency (583)	The core section. The paper is an extension of this.
12.4.5	Shared Caches (596)	Background.
12.4.6	Coherent Private Caches (596)	Extended to full MESI/MOESI and directory protocols.
12.4.7	Implementing a Memory Consistency Model* (603)	Direct antecedent of the speculative-SC discussion.
Supporting		
10.4	Pipeline Hazards (420)	Reordering and dependences.
10.9	Performance Metrics (463)	Benchmarking framework.
10.11.4	Out-of-Order Pipelines (492)	Speculation and the squash mechanism.
11.2	Caches (517)	Private caches, non-blocking misses.
11.3	The Memory System (532)	Latency figures motivating the store buffer.
12.2.2	Shared Memory vs Message Passing (571)	Framing.
12.2.3	Amdahl's Law (576)	Scaling discussion.
12.6	Interconnection Networks (620)	Coherence latency in the simulator.
App. A	Cortex-A8/A15, Sandy Bridge (734–746)	Real-machine case studies.

Table 1: Mapping of the topic onto the prescribed text. * denotes a section marked advanced.

Additional texts. Because §12.4.3 treats only sequential consistency and the relaxed models are outside the scope of the prescribed text, three further sources carry the technical weight of the paper. All are fully specified, with reproduced tables of contents, in Chapter 7 of the paper draft.

- S. R. Sarangi, *Next-Gen Computer Architecture: Till the End of Silicon*, White Falcon 2023, ISBN 8119510143 [21]. Chapter 9, *Multicore Systems: Coherence, Consistency and Transactional Memory*, runs to roughly 118 pages and is the source of the formalism used throughout.
- V. Nagarajan, D. J. Sorin, M. D. Hill and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., Springer 2020 [19]. The standard reference on the subject.
- Vendor architecture manuals (Intel SDM Vol. 3A; Arm ARM Ch. B2; RISC-V RVWMO), cited in preference to any textbook wherever the paper states what a particular instruction set guarantees.

1.3 Advisor

I will consult **Prof. [Sumantra Dutta Roy / Subrat Kar — SELECT ONE]** as advised.

Zotero library `ELL305-MemoryConsistency` has been created and will be shared with Prof. Subrat Kar. Bibliography is exported to `refs.bib` and rendered via BibTeX.

1.4 Progress

Summary

WP	Work package	Target	Complete
WP1	Foundations: prescribed textbook	31 Aug	55%
WP2	Advanced sources	14 Sep	30%
WP3	Literature survey and Zotero	21 Sep	35%
WP4	Writing: formal, historical, methodological chapters	24 Aug	90%
WP5	Litmus experiments on real hardware	28 Sep	10%
WP6	Simulator design and evaluation	26 Oct	0%
WP7	Verification	02 Nov	0%
WP8	Final assembly and appendices	11 Nov	0%
Overall			28%

Table 2: Work package status as of 24 August 2026.

Note to the author: every percentage in this section and in the Gantt chart must be corrected to reflect what has *actually* been done before submission. Figures that overstate progress will be contradicted by the next draft and are not worth the marks they might gain.

Gantt Chart

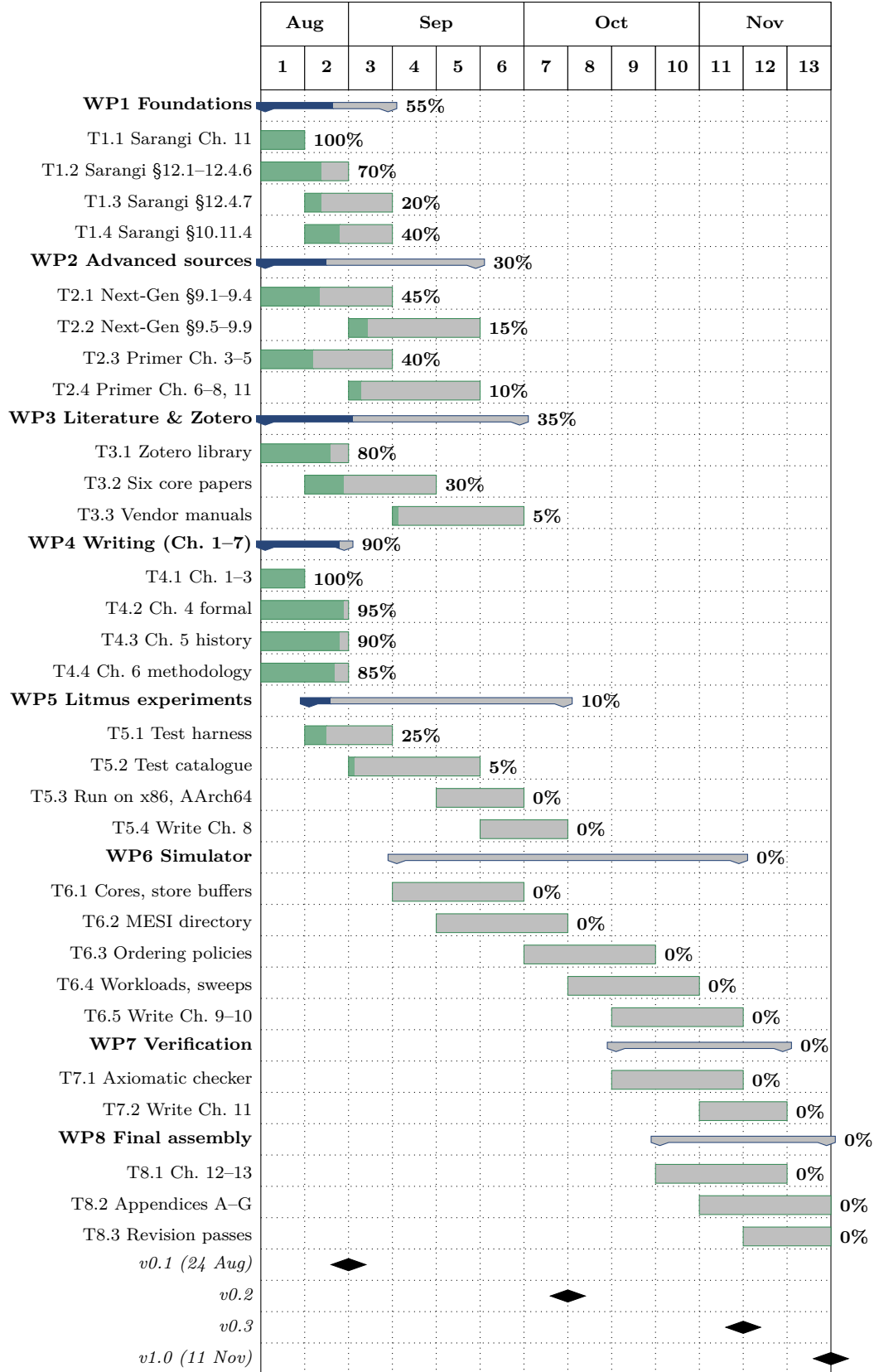


Figure 1: Project schedule. Week 1 commences 17 August 2026; week 13 commences 9 November 2026. Shaded portions of each bar indicate completion. Milestone dates for v0.2 and v0.3 are estimates pending confirmation.

Work Package Detail

WP1 — Foundations (55%).

- **T1.1** Sarangi [22] Ch. 11, *The Memory System*, §11.1–11.3 (pp. 505–560). **100%**. Cache organisation, locality, and the latency figures that motivate the store buffer.
- **T1.2** Sarangi Ch. 12, §12.1–12.4.6 (pp. 565–603). **70%**. §12.4.2 and §12.4.3 read closely; §12.4.6 in progress.
- **T1.3** Sarangi §12.4.7, *Implementing a Memory Consistency Model* (pp. 603–608). **20%**. Begun; requires §12.4.6 first.
- **T1.4** Sarangi §10.11.4, *Out-of-Order Pipelines* (pp. 492–505). **40%**. Needed for the squash mechanism central to Q3.

WP2 — Advanced sources (30%).

- **T2.1** Sarangi [21] §9.1–9.4. **45%**. §9.4 supplies the execution-witness and access-graph notation adopted throughout the paper, chosen over the alternatives for continuity with the course.
- **T2.2** Sarangi §9.5–9.9 (coherence protocols, memory models, race detection, transactional memory). **15%**.
- **T2.3** Nagarajan et al. [19] Ch. 3–5 (SC, TSO, relaxed models). **40%**.
- **T2.4** Nagarajan et al. Ch. 6–8, 11 (coherence protocols, specification and validation). **10%**.

WP3 — Literature and Zotero (35%).

- **T3.1** Zotero library created; 22 entries filed with PDFs. **80%**. Exported to `refs.bib`, under version control.
- **T3.2** Six papers identified as load-bearing for the argument and to be read in full rather than consulted: Lamport [16]; Dubois, Scheurich and Briggs [9]; Adve and Hill [2]; Adve and Gharachorloo [1]; Gniady, Falsafi and Vijaykumar [11]; Alglave, Maranget and Tautschnig [3]. **30%**.
- **T3.3** Vendor manuals [15, 4, 20]. **5%**.

Further references filed and cited: Gharachorloo et al. [10]; Hill [14]; Manson, Pugh and Adve [17]; Boehm and Adve [6]; Ceze et al. [7]; Blundell, Martin and Wenisch [5]; Marino et al. [18]. Supporting texts: Hennessy and Patterson [12]; Culler, Singh and Gupta [8]; Herlihy et al. [13].

WP4 — Writing (90%). A 42-page draft of the paper proper exists in the repository (`main.tex`), comprising:

Ch.	Title	State
1	Introduction — motivating example, problem statement, contributions	drafted
2	Background — memory wall, caches, store buffer, out-of-order	drafted
3	Cache Coherence, and What It Does Not Solve	drafted
4	The Design Space of Memory Consistency Models	drafted
5	Historical Evolution	drafted
6	Methodology	drafted
7	Source Texts and Their Coverage	drafted

Table 3: State of the paper draft. Seven chapters, 42 pages, five TikZ figures, compiling without errors or undefined references.

All figures are drawn in TikZ per the course requirement. The document is built with `latexmk`; `main.pdf` is deliberately excluded from version control and attached instead to tagged releases.

WP5 — Litmus experiments (10%). Design complete and documented in Chapter 6 of the paper draft; implementation begun. The harness must avoid `pthread_barrier`, whose internal fences suppress the window under observation, in favour of a sense-reversing spin barrier built from relaxed atomics. Shared variables are padded to distinct cache lines to prevent false sharing from confounding the coherence behaviour.

WP6–WP8 (0%). Not started; scheduled per the Gantt chart. Designs for WP6 (simulator) and WP7 (verification) are specified in Chapter 6 of the paper draft, stated in advance so that the methodology can be assessed independently of the results.

1.5 Risks and Mitigations

Risk	Sev.	Mitigation
Litmus tests yield zero observations, making Ch. 8 empty	Med	Vary thread placement, optimisation level and inserted delay; a null result with the parameter sweep documented is itself reportable.
Simulator scope creep consumes the schedule	High	Feature list frozen after v0.2. Purpose-built simulator is the baseline; Tejas [21] evaluated in parallel as a stretch goal only, never a dependency.
No AArch64 hardware available for Q2	Med	Single-board computer, Android device under Termux, or institutional server; failing all three, Q2 narrows to x86 and the limitation is stated.
Topic requires material beyond the course	Low	Sarangi’s companion volume [21] is pitched at exactly this level and is by the same author.

1.6 Plan to v0.2

1. Complete WP1 (T1.2–T1.4) and advance WP2 to **70%**.
2. Read the six load-bearing papers in full (T3.2 to **100%**).
3. Implement the litmus harness and the full test catalogue; obtain observation counts on x86 and, if hardware permits, AArch64.
4. Draft Chapter 8 (Results I) from those measurements.

5. Begin WP6: cores, store buffers and load queue.
6. Insert the measured SB violation count into §1.1 of the paper, so that the opening example is a measurement rather than an assertion.

1.7 Repository

All work is under version control at

<https://github.com/y4shv/ELL305-term-paper> (private; access granted to the instructor on request).

Draft #1 corresponds to tag v0.1. `draft1.tex` builds this progress report; `main.tex` builds the paper draft described in WP4. `refs.bib` is exported from Zotero.

References

- [1] Sarita V. Adve and Kourosh Gharachorloo. Shared memory consistency models: A tutorial. *IEEE Computer*, 29(12):66–76, 1996.
- [2] Sarita V. Adve and Mark D. Hill. Weak ordering — a new definition. In *Proc. 17th International Symposium on Computer Architecture (ISCA)*, pages 2–14, 1990.
- [3] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems*, 36(2):7:1–7:74, 2014.
- [4] Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture*. Chapter B2: The AArch64 Application Level Memory Model.
- [5] Colin Blundell, Milo M. K. Martin, and Thomas F. Wenisch. InvisiFence: Performance-transparent memory ordering in conventional multiprocessors. In *Proc. 36th International Symposium on Computer Architecture (ISCA)*, pages 233–244, 2009.
- [6] Hans-J. Boehm and Sarita V. Adve. Foundations of the C++ concurrency memory model. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 68–78, 2008.
- [7] Luis Ceze, James Tuck, Pablo Montesinos, and Josep Torrellas. BulkSC: Bulk enforcement of sequential consistency. In *Proc. 34th International Symposium on Computer Architecture (ISCA)*, pages 278–289, 2007.
- [8] David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. *Parallel Computer Architecture: A Hardware/Software Approach*. Morgan Kaufmann, 1998.
- [9] Michel Dubois, Christoph Scheurich, and Fayé Briggs. Memory access buffering in multiprocessors. In *Proc. 13th International Symposium on Computer Architecture (ISCA)*, pages 434–442, 1986.
- [10] Kourosh Gharachorloo, Daniel Lenoski, James Laudon, Phillip Gibbons, Anoop Gupta, and John Hennessy. Memory consistency and event ordering in scalable shared-memory multiprocessors. In *Proc. 17th International Symposium on Computer Architecture (ISCA)*, pages 15–26, 1990.
- [11] Chris Gniady, Babak Falsafi, and T. N. Vijaykumar. Is SC + ILP = RC? In *Proc. 26th International Symposium on Computer Architecture (ISCA)*, pages 162–171, 1999.

- [12] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.
- [13] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.
- [14] Mark D. Hill. Multiprocessors should support simple memory consistency models. In *IEEE Computer*, volume 31, pages 28–34, 1998.
- [15] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 3A: System Programming Guide*. Chapter 9: Memory Ordering.
- [16] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.
- [17] Jeremy Manson, William Pugh, and Sarita V. Adve. The Java memory model. In *Proc. 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 378–391, 2005.
- [18] Daniel Marino, Abhayendra Singh, Todd Millstein, Madanlal Musuvathi, and Satish Narayanasamy. A case for an SC-preserving compiler. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 199–210, 2011.
- [19] Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood. *A Primer on Memory Consistency and Cache Coherence*. Synthesis Lectures on Computer Architecture. Springer Nature, 2nd edition, 2020.
- [20] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Chapter on RVWMO Memory Consistency Model.
- [21] Smruti R. Sarangi. *Next-Gen Computer Architecture: Till the End of Silicon*. White Falcon Publishing, 2023. Chapter 9: Multicore Systems — Coherence, Consistency and Transactional Memory.
- [22] Smruti R. Sarangi. *Basic Computer Architecture*. White Falcon Publishing, version 3.09 edition, 2025. Available at <https://srsarangi.github.io/archbook/>.